Automatic Log Template Extraction Using Large Language Models

Shan Ali (Student ID: 300332700)
Hossein Yousefizadeh (Student ID: 300361320)
Kasra Rasinoujehdehi (Student ID: 300386444)

April 23, 2024

Abstract

Software-intensive systems typically produce a massive amount of logs that record different executions of these systems. These logs are in unstructured form which require further preprocessing to extract meaningful log templates by converting them into a structured form to facilitate the system monitoring and anomaly detection. In this project, we investigate the efficacy of Large Language Models (LLMs) for log template extraction (LTE) using two different techniques: In-Context Learning (ICL) and Prefix Tuning (PT). We perform experiments on four commonly adopted benchmark log-based datasets: HDFS, Hadoop, BGL, and Thunderbird, employing metrics such as Rouge, BLEU, Parsing Accuracy (PA), Precision Template Accuracy (PTA), and Recall Template Accuracy (RTA). The results demonstrate that while ICL provides a reliable baseline across different models, PT distinctively outperforms ICL in most metrics across all datasets. PT's refined approach to tuning model attention mechanisms significantly enhances the LTE performance on limited data scenarios, notably improving the capacity of language models to parse and structure complex log data accurately. This study not only validates the robust capabilities of LLMs in structuring complex log data but also underscores the strategic importance of choosing the right learning technique for specific applications in automated log analysis.

1 Introduction

Software systems commonly generate extensive volumes of logs through log files that record various executions within these systems. A log file usually contains log messages from both normal and abnormal executions of a system (e.g., starting or stopping a process, allocating system resources, system crash report, granting user permissions, etc.), that can be used for troubleshooting, service maintenance, or security purposes. Each log message may contain information about the timestamp it was recorded, the level (error, warning, info, etc.) of the log message, and the corresponding textual description of the system operation.

For example, Table 1 shows a log file instance with three log messages. Each log message is defined with a timestamp at which it occurred and the corresponding message. Usually, these raw logs are in an unstructured form so the first step is to automatically extract log templates and convert these logs into a structured format for further analysis. Template extraction (conventionally known as log parsing) is a process of grouping log messages that share the same textual structure.

Table 1: Example of a log file

Timestamp	**Log message**	**Log template**
10 : 05 : 12	send pkt_436 for block 123	send $< * >$ for block $< * >$
10 : 05 : 15	receive pkt_436 for block 123	receive $< * >$ for block $< * >$
10 : 05 : 16	send pkt_111 for block 124	send $< * >$ for block $< * >$

As these produced logs are massive in size, it is not possible for system engineers to manually extract templates to identify the abnormal behavior of the system [1]. In Table 1, we also show the extracted log templates (column Log template) for each log message. However, please note that a log file does not contain this information and one relies on LTE techniques for this purpose. Therefore, numerous data-driven log template extraction (LTE) techniques have been proposed in the literature. They usually fall into two categories: i) traditional LTE and ii) deep learning (DL)–based LTE techniques. The traditional LTE techniques have further subcategories including clustering-based, tree-based, and dictionary-based techniques. Clustering-based LTE techniques (LKE [2], LogSig [3], LogMine [4], SHISO [5], LenMa [6]) extract log templates by computing the similarity among different log messages and grouped them into different clusters. However, these LTE techniques require a threshold to determine if the log message belongs to a cluster which can be time-consuming and labor-intensive. Another subcategory of clustering-based LTE techniques relies on the frequency of the constant parts that appear within log messages. For example, SLCT [7], LFA [8] and LogCluster [9], first count the constant parts from all log messages and then extract templates by dividing log messages into different groups. Tree-based LTE techniques (Drain [11] and Spell [10]) extract log templates by means of the tree structure. Drain builds a fixed depth (a tuneable parameter) tree from the input log messages and splits them into different nodes to extract different templates. Drain also depends on the similarity threshold (a user-defined parameter) while building the tree. However, it is time-extensive to obtain optimal results by configuring these parameters. Dictionary-based LTE techniques construct dictionaries from the log messages to identify the constant and variable parts within log messages leading to the extraction of log templates. Logparse [12] first constructs a dictionary and then trains a word classifier to predict log templates. Logram [13] is also a dictionary-based LTE technique. However, it extracts log templates by constructing n-gram dictionaries from the constant and variable parts of the log messages.

DL-based LTE techniques (UniParser [14], SemParser [15], LogPPT [19]) have been recently proposed to extract log templates by means of deep learning techniques such as BiLSTM. However, the existing LTE techniques still suffer from limited robustness, leading to unsatisfactory accuracy [19, 24]. For example, the traditional LTE techniques (e.g., Drain [11]) rely on manually designed human-crafted features or heuristics (i.e., regular expressions, similarity threshold, etc.) to use as patterns for extracting templates. However, they often fall short in log messages where the template does not match with the input patterns. Similarly, the DL-based LTE techniques (e.g., LogPPT [19]) may not perform well on the unknown log patterns when there is a large data diversity and the training set lacks different variations within logs.

2 Background

In recent decade, natural language processing (NLP) has witnessed a significant advancements propelled by the emergence of large language models (LLMs). These models, such as Generative Pre-trained Transformer (GPT [16]), have shown remarkable capabilities on a diverse range of tasks such as understanding, text generation, and parsing natural language. However, one of the key challenges in NLP has been the effective utilization of these models for specific downstream tasks while maintaining their generality.

Log template extraction is a crucial task in software development, system monitoring and data analytics. It involves the extraction of structured information from unstructured or semi-structured log files to facilities system engineers understand the cause of the system failure and detect anomalies that deviate from the normal system behavior. Traditional methods often rely on predefined rules or regular expressions, which are time-consuming and labor-intensive to create and maintain, and may not generalize well to diverse log formats. To tackle these challenges, in this report, we adopt and assess different LLMs to effectively extract log templates from different log-based datasets (i.e., HDFS [21], Hadoop [23], BGL [22], Thunderbird [22]). However, fine-tuning these LLMs on custom datasets demand high computational cost due to their large number of parameters. To alleviate this, we adopt Prefix Tuning (PT) [25] and In-Context Learning (ICL) to efficiently utilize these models for LTE. In the following, we briefly discuss both the techniques and the LLMs adopted in this report.

Prefix Tuning

Prefix Tuning (PT) [25] is a specialized form of transfer learning where the model is fine-tuned using task-specific prefixes. For log template extraction, we will leverage PT to tailor the LLMs specifically to the nuances of log data. This approach allows us to impart domain-specific knowledge to the models, enhancing their ability to understand and extract meaningful log templates.

In-Context Learning

While PT takes the forefront in our approach, ICL can still be a noticeable alternative. By providing examples and prompts within the context of log data, we reinforce the model's understanding of the structure and semantics of log messages. The project includes a comparative analysis of the performance of PT and ICL, providing insights into the effectiveness of these two approaches in enhancing LTE accuracy.

Large Language Models

To address the above challenges in existing LTE techniques, we propose a new approach to effectively extract templates from log messages using large language models (LLMs). Large Language models are state-of-the-art natural language processing models that have demonstrated remarkable proficiency in understanding and generating human-like text. These models, often pre-trained on massive amounts of text data, capture intricate language patterns and semantic information. LLMs have exhibited exceptional language understanding and generation capabilities. These models can be fine-tuned for specific tasks, or guided by carefully written prompts, making them versatile tools for various natural language processing applications. In the context of log template extraction, LLMs offer the potential to discern complex patterns within unstructured log data, aiding in the creation of meaningful templates. For this project, we consider the following LLMs:

- **GPT-2**: GPT-2, or Generative Pre-trained Transformer 2, is an advanced natural language processing model developed by OpenAI [16]. Known for its large-scale architecture and impressive language generation capabilities, GPT-2 has been widely utilized in various applications, showcasing its ability to generate coherent and contextually relevant text based on given prompts.

- **InCoder**: InCoder is a unified generative model trained on a large corpus of code [17]. It has been shown commendable performance across various code-related tasks, such as automated log generation [18].

- **T5**: T5 [27] is an encoder-decoder LLM model developed by Google pre-trained on a multi-task mixture of both unsupervised and supervised tasks. It shows a good performance on a wide range of tasks where each task is first converted into a text-to-text format. T5 prepends different prefix to the input corresponding to each separate task to learn better data representation specific to each task, e.g., for summarization: summarize, etc. As T5 shares similar methodology of prepending text tokens to the input with PT, we adopted this model to extract templates from the log-based datasets.

- **BART**: BART [28] is developed by Facebook and adopts a standard seq2seq/machine translation architecture with a bidirectional encoder (i.e.,

BERT) and a left-to-right decoder (i.e., GPT). During pre-training phase, the original sentences are randomly shuffled and filled-in with a novel in-filling scheme where spans of text are replaced with a single mask token. BART shows a comparable performance with RoBERTa [29] and outperforms the state-of-the-art techniques when fine-tuned on variety of tasks such as abstractive dialogue, question-answering and summarization. We adopted this model as it was considered as one of the baselines in the original paper of Pref-x-tuning due to its effectiveness on the complex tasks (like summarization).

3 Methodology

This project employs Large Language Models (LLMs) including GPT-2, InCoder, T5 and BART to extract log templates from system logs, utilizing two distinct techniques: ICL and PT. The methodology is implemented on four benchmark datasets: HDFS, Hadoop, BGL, and Thunderbird, allowing for a comprehensive comparative analysis between the two techniques across different models and datasets.

In-Context Learning

ICL leverages the ability of LLMs to learn directly from examples provided in the prompt without explicit parameter updates. For this project, ICL is implemented by appending multiple log template extraction examples directly into the input prompt. This allows the model to infer the task from the context provided by these examples. The following figures 1, 2 illustrate the python implementation of ICL using GPT-2 for log template extraction. Each model-generated prediction is decoded and processed to extract log templates marked between the `<START>` and `<END>` tags, ensuring that the output aligns with the expected format.

```python
def prep_examples(train, num_examples):
    examples = train.sample(num_examples).to_dict(orient='records')
    examples_str = ""
    for example in examples:
        examples_str += f"""

        Message: {example['message']}
        Template: <START {example['template']} <END>
        """
    return examples_str
```

Figure 1: Python implementation to prepare examples for In-Context Learning.

```python
def gpt2_predict(input):
    examples = prep_examples(train, num_examples)
    final_prompt = base_prompt + examples + f"\n\nMessage: {input}\n"
    prompt_token_size = len(tokenizer.encode(final_prompt))
    inputs = tokenizer.encode(
        final_prompt,
        return_tensors="pt",
        add_special_tokens=True
    )
    with torch.no_grad():
        outputs = model.generate(
            inputs,
            max_length=prompt_token_size + 100,
            num_return_sequences=1,
            pad_token_id=tokenizer.eos_token_id,
            temperature=0.01,
            do_sample=True
        )
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)
    output = response.split('<START>')[1].split('<END>')[0].strip()
    return output
```

Figure 2: Python implementation to predict log templates using In-Context Learning.

Prefix Tuning

Prefix tuning is a lightweight alternative to fine-tuning Natural Language Generation (NLG) tasks. It adds a series of task-specific vectors (called the prefix) to the input sequence that can be learned and optimized while keeping the pretrained language model frozen. These prefixes are prepended to the input sequence before feeding it to the model. According to the original PT paper [25], PT can achieve comparable performance to fine-tuning with the adjustment of only 0.1% of the model parameters.

The intuition of the PT relies on prompting and is based on the assumption that a proper context can steer the performance of LLM without changing its model parameters. For example; if the LLM is expected to generate a word (e.g., Obama), the training corpus should contain all the common collocations as context (e.g., Barack). To adapt PT for extracting log templates, the goal is to find a prompt that can steer the model for generating entire logs templates from the log messages, not just single words. In pursuit of this goal, we repurpose PT as a summarization task to generate log templates from raw log messages.

Motivated by the capabilities of PT on summarization tasks (as illustrated in Figure 3a), we repurpose it for the extraction of log templates (as demonstrated in Figure 3b). This approach harnesses the inherent capabilities of PT to distill meaningful information from raw log data similar to summarization task, enabling the generation of comprehensive log templates that capture the essence of the underlying log messages.

(a) Summarization Example (b) Log Template Extraction Example

Figure 3: Prefix tuning for Summarization and Log Template Extraction

Dataset and Evaluation

The datasets used in this study are pre-processed to align with the input requirements of the LLMs. Each dataset is split into training and test sets, with the test set used to evaluate the model's performance. The effectiveness of the extracted templates is assessed using the Rouge metric, providing a quantitative measure of the similarity between the generated templates and the ground truth. Additionally, for more granular analysis and comparison, we utilize task-specific metrics such as Parsing Accuracy (PA), Precision Template Accuracy (PTA), and Recall Template Accuracy (RTA) [26], which provide further insight into the precision and recall of the template extraction capabilities of each model. These metrics focus on evaluating the models' performance in terms of accuracy, computational efficiency, and their ability to generalize across different log formats. Results from this analysis are crucial for identifying the most effective method for log template extraction in real-world applications.

4 Results

In this section, we present the performance of GPT-2 and InCoder models on the task of log template extraction using ICL and PT techniques. The evaluation is conducted on the HDFS, Hadoop, BGL, and Thunderbird datasets, utilizing metrics such as Rouge-1, Rouge-2, Rouge-L, BLEU, PA, PTA, and RTA.

As depicted in Table 2, we first report the results obtained by applying ICL using different LLMs (GPT-2, InCoder, T5 and Bart) followed by the results of PT (see Table 3) using T5 and Bart. The reason for not considering the other two models (GPT-2 and InCoder) using PT is due to the model's incompatibility with PT. There are other options for efficient fine-tuning such as LoRA [30] that support a wide range of LLMs including GPT-2 and Incoder that we plan to consider in our future work.

For a fair comparison, we present the results obtained by both techniques (PT and ICL) using the same LLMs (T5 and BART) side by side in Table 4.

Table 2: ICL Results for GPT-2, InCoder, T5, and BART Models

Dataset	Rouge-1	Rouge-2	Rouge-L	BLEU	PA	PTA	RTA
GPT-2							
HDFS	84.09	78.29	83.96	0.77	0.66	0.13	0.73
Hadoop	85.53	79.57	85.53	0.74	0.50	0.19	0.39
BGL	87.31	80.07	87.29	0.61	0.59	0.21	**0.39**
Thunderbird	90.74	87.58	90.74	**0.86**	0.79	**0.31**	**0.39**
InCoder							
HDFS	**94.05**	**90.99**	**93.91**	**0.91**	**0.81**	**0.22**	**1.00**
Hadoop	**88.81**	**84.64**	**88.81**	**0.85**	**0.69**	**0.21**	**0.50**
BGL	**89.82**	**83.31**	**89.73**	**0.69**	**0.64**	0.21	0.38
Thunderbird	**92.26**	**89.51**	**92.19**	0.81	**0.80**	0.29	0.34
T5							
HDFS	43.86	21.96	43.67	0.10	0.01	0.01	0.09
Hadoop	71.42	56.39	71.42	0.45	0.14	0.05	0.17
BGL	62.85	54.41	62.67	0.23	0.33	0.08	0.17
Thunderbird	66.40	61.78	66.40	0.31	0.26	0.05	0.18
BART							
HDFS	15.46	5.09	14.53	0.03	0.00	0.00	0.00
Hadoop	11.52	4.65	11.33	0.02	0.00	0.00	0.00
BGL	18.86	14.34	18.77	0.02	0.14	0.03	0.04
Thunderbird	17.06	10.59	17.06	0.08	0.09	0.01	0.03

Table 3: PT Results for T5 and BART Models

Dataset	Rouge-1	Rouge-2	Rouge-L	BLEU	PA	PTA	RTA
T5							
HDFS	69.37	55.25	68.80	0.39	0.06	**0.02**	**0.33**
Hadoop	**70.65**	**59.75**	**70.62**	**0.40**	**0.04**	**0.03**	**0.16**
BGL	**72.70**	**60.83**	**72.38**	**0.31**	**0.31**	**0.07**	**0.20**
Thunderbird	**78.98**	**69.88**	**78.50**	**0.66**	**0.35**	**0.03**	**0.10**
BART							
HDFS	**87.56**	**81.37**	**87.28**	**0.54**	**0.10**	0.01	0.09
Hadoop	37.40	15.33	36.31	0.05	0.00	0.00	0.00
BGL	38.89	13.02	37.88	0.08	0.00	0.00	0.00
Thunderbird	47.00	0.00	47.00	0.00	0.00	0.00	0.00

Table 4: Comparison of ICL and PT Results for T5 and BART Models

Dataset	Rouge-1		Rouge-2		Rouge-L		BLEU	
	ICL	PT	ICL	PT	ICL	PT	ICL	PT
T5								
HDFS	43.86	69.37	21.96	55.25	43.67	68.80	0.10	0.39
Hadoop	71.42	70.65	56.39	59.75	71.42	70.62	0.45	0.40
BGL	62.85	72.70	54.41	60.83	62.67	72.38	0.23	0.31
Thunderbird	66.40	78.98	61.78	69.88	66.40	78.50	0.31	0.66
BART								
HDFS	15.46	87.56	5.09	81.37	14.53	87.28	0.03	0.54
Hadoop	11.52	37.40	4.65	15.33	11.33	36.31	0.02	0.05
BGL	18.86	38.89	14.34	13.02	18.77	37.88	0.02	0.08
Thunderbird	17.06	47.00	10.59	0.00	17.06	47.00	0.08	0.00

Dataset	PA		PTA		RTA	
	ICL	PT	ICL	PT	ICL	PT
T5						
HDFS	0.01	0.06	0.01	0.02	0.09	0.33
Hadoop	0.14	0.04	0.05	0.03	0.17	0.16
BGL	0.33	0.31	0.08	0.07	0.17	0.20
Thunderbird	0.26	0.35	0.05	0.03	0.18	0.10
BART						
HDFS	0.00	0.10	0.00	0.01	0.00	0.09
Hadoop	0.00	0.00	0.00	0.00	0.00	0.00
BGL	0.14	0.00	0.03	0.00	0.04	0.00
Thunderbird	0.09	0.00	0.01	0.00	0.03	0.00

5 Discussion

In this section, we discuss and evaluate the performance of ICL and PT techniques for log template extraction and present our findings. A side-by-side comparison of ICL and PT results for the T5 and BART models reveals interesting dynamics between these techniques.

Comparative Performance of ICL and PT

In the comparative analysis between ICL and PT, the PT approach emerges as the superior technique across most key performance metrics. The PT method exhibits a consistent outperformance over ICL in all datasets for the T5 model. This is particularly evident in the HDFS dataset, where PT achieves a significant boost in Rouge-L scores compared to ICL, underscoring PT's enhanced ability to grasp the extensive structure of log messages with T5. Even the BART model, which showed modest results with ICL, registers a notable surge in performance

when utilizing PT, as seen in the marked improvement of Rouge scores in the HDFS dataset. Such advancements imply that PT's refined optimization of the language model's focus mechanisms is exceptionally advantageous. The findings clearly demonstrate that PT not only transcends the solid foundation set by ICL but also substantially elevates the capabilities of both T5 and BART in the domain of log template extraction. The substantial advantages provided by PT underscore its potential as the preferred approach for optimizing log template accuracy and reliability in language models.

Model Comparison

In the ICL setting, InCoder stands out significantly, boasting the highest scores in almost all metrics, indicative of its sophisticated understanding of log structures, likely due to its domain-specific pre-training. GPT-2 also shows strong performance, but it does not reach the high bar set by InCoder, which may be attributed to GPT-2's more generalized pre-training. T5 and BART models exhibit moderate performance under ICL, with BART having particular difficulty in achieving competitive scores, suggesting a potential mismatch between the model's pre-training and the structure of log messages.

When we focus on the PT approach, T5 shows a commendable increase in its performance, especially in terms of BLEU score improvements in datasets such as Hadoop and Thunderbird. This demonstrates T5's ability to benefit from the PT approach, adapting more effectively to the task of log template extraction. On the other hand, BART's enhancement through PT is less consistent. While there is an increase in some metrics for specific datasets, such as the HDFS dataset, BART does not maintain this improvement across all datasets. This inconsistency suggests that BART may require more fine-tuning or a different approach to PT to realize its full potential in LTE tasks.

This analysis underscores the importance of technique selection in leveraging the strengths of different language models for specific natural language processing tasks.

6 Conclusion

The comprehensive analysis conducted in this study has underscored the distinct advantages of PT over ICL in the context of log template extraction tasks on limited data scenarios. While ICL establishes a robust baseline, offering consistent and reliable performance across various models and datasets, PT emerges as the superior technique, particularly in its application with the T5 and BART models on datasets that contain limited labelled samples. The findings also reveal that PT significantly enhances task-specific metrics, such as PA, PTA, RTA. These improvements are notable in complex datasets where understanding the nuanced structure of log messages is critical. This study paves the way for further exploration into refining PT techniques and potentially combining elements of both PT and ICL to harness their respective strengths, ultimately

contributing to more sophisticated natural language processing tools in the field of system maintenance and security. The results reported in this study highlight that LLMs can be suitable and reliable option over the traditional log template extraction techniques for extracting different log patterns and templates on limited data to facilitate the system monitoring and log analysis.

References

[1] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," ACM computing surveys (CSUR), vol. 54, no. 6, pp. 1–37, 2021.

[2] Q. Fu, J.-G. Lou, Y. Wang, and J. Li, "Execution anomaly detection in distributed systems through unstructured log analysis," in Proc. 9th IEEE Int. Conf. Data Mining, 2009, pp. 149–158.

[3] M. Mizutani, "Incremental mining of system log format," in Proc. IEEE Int. Conf. Serv. Comput., 2013, pp. 595–602.

[4] H. Hamooni, B. Debnath, J. Xu, H. Zhang, G. Jiang, and A. Mueen, "LogMine: Fast pattern recognition for log analytics," in Proc. 25th ACM Int. Conf. Informat. Knowl. Manage., 2016, pp. 1573–1582.

[5] K. Q. Zhu, K. Fisher, and D. Walker, "Incremental learning of system log formats," ACM SIGOPS Oper. Syst. Rev., vol. 44, no. 1, pp. 85–90, 2010.

[6] K. Shima, "Length matters: Clustering system log messages using length of words," 2016, arXiv:1611.03213.

[7] R. Vaarandi, "A data clustering algorithm for mining patterns from event logs," in Proc. 3rd IEEE Workshop IP Oper. Manage., 2003, pp. 119–126.

[8] M. Nagappan and M. A. Vouk, "Abstracting log lines to log event types for mining software system logs," in Proc. 7th IEEE Work. Conf. Mining Softw. Repositories, 2010, pp. 114–117.

[9] R. Vaarandi and M. Pihelgas, "LogCluster-a data clustering and pattern mining algorithm for event logs," in Proc. 11th Int. Conf. Netw. Serv. Manage., 2015, pp. 1–7

[10] M. Du and F. Li, "Spell: Streaming parsing of system event logs," in Proc. 16th Int. Conf. Data Mining, 2016, pp. 859–864.

[11] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in Proc. IEEE Int. Conf. Web Serv., 2017, pp. 33–40.

[12] W. Meng et al., "LogParse: Making log parsing adaptive through word classification," in Proc. 29th Int. Conf. Comput. Commun. Netw., 2020, pp. 1–9.

[13] H. Dai, H. Li, C. S. Chen, W. Shang, and T.-H. Chen, "Logram: Efficient log parsing using n-gram dictionaries," IEEE Trans. Softw. Eng., vol. 48, no. 3, pp. 879–892, Mar. 2022.

[14] Y. Liu, X. Zhang, S. He, H. Zhang, L. Li, Y. Kang, Y. Xu, M. Ma, Q. Lin, Y. Dang et al., "Uniparser: A unified log parser for heterogeneous log data," in Proceedings of the ACM Web Conference 2022, 2022, pp.1893–1901.

[15] Y. Huo, Y. Su, B. Li, and M. R. Lyu, "Semparser: A semantic parser for log analysis," arXiv preprint arXiv:2112.12636, 2021.

[16] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners.

[17] Fried, D., Aghajanyan, A., Lin, J., Wang, S., Wallace, E., Shi, F., Zhong, R., Yih, W., Zettlemoyer, L., & Lewis, M. (2022). InCoder: A Generative Model for Code Infilling and Synthesis. https://arxiv.org/abs/2204.05999v3

[18] Li, Y., Huo, Y., Jiang, Z., Zhong, R., He, P., Su, Y., & Lyu, M. R. (2023). Exploring the Effectiveness of LLMs in Automated Logging Generation: An Empirical Study. https://arxiv.org/abs/2307.05950v1

[19] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," arXiv preprint arXiv:2302.07435, 2023.

[20] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, "Tools and benchmarks for automated log parsing," in 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, 2019, pp. 121–130.

[21] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles,pp. 117–132, 2009

[22] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in 37th annual IEEE/IFIP international conference on dependable systems and networks (DSN'07). IEEE, pp. 575–584, 2007

[23] Lin, Q., Zhang, H., Lou, J.-G., Zhang, Y., Chen, X.: Log clustering based problem identification for online service systems. In: Proceedings of the 38th International Conference on Software Engineering Companion, pp. 102–111 (2016)

[24] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, "Tools and benchmarks for automated log parsing," in 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, 2019, pp. 121–130.

[25] Li, X. L., & Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. arXiv preprint arXiv:2101.00190.

[26] Zanis Ali Khan, Donghwan Shin, Domenico Bianculli, and Lionel Briand. 2022. Guidelines for Assessing the Accuracy of Log Message Template Identification Techniques. In Proceedings of the 44th International Conference on Software Engineering (ICSE'22). ACM.

[27] Raffel, Colin, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. "Exploring the limits of transfer learning with a unified text-to-text transformer." Journal of machine learning research 21, no. 140 (2020): 1-67.

[28] Lewis, Mike, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension." arXiv preprint arXiv:1910.13461 (2019).

[29] Liu, Yinhan, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. "Roberta: A robustly optimized bert pretraining approach." arXiv preprint arXiv:1907.11692 (2019).

[30] Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "Lora: Low-rank adaptation of large language models." arXiv preprint arXiv:2106.09685 (2021).